

清华大学本科生考试试题纸		
考试课程：操作系统	时间：2026年4月 26 日下午18:30~21:00	A卷
姓名：_____	班级：_____	学号：_____
答卷 注意 事项：	1. 答题前在答卷纸每一页左上角写明姓名、班级、学号、页码和总页数。	
	2. 在答卷纸上答题，写明具体题号。完成后，请将试题和答卷一起交回。	
	3. 请注意回答所有试题。本试卷共7页，最后一页的空白页可做草稿纸。	

一、判断题（15分）

1. 在QEMU RISC-V Virt Machine 的模拟加电启动执行路径中，处理器执行的第一条指令来自BIOS固件，如 OpenSBI / RustSBI，它们通常先于操作系统内核获得控制权，并完成最早期的硬件与执行环境准备。
2. 在协作式调度多任务模型中，一个不主动让出 CPU 的用户态死循环不会阻塞其他任务的推进。
3. 采用轮转调度（Round Robin, RR）时，如果时间片设置得无限大，该算法在逻辑上等同于先来先服务（FCFS）算法。
4. 在最短剩余时间算法（SRT）下，在新作业到达且其剩余时间更短时，将抢占当前正在运行的作业。
5. 最高响应比优先（HRRN）的响应比可写为（阻塞等待时间 + 执行时间）/ 执行时间，响应比高的任务先执行。
6. 在分页系统中，虚拟地址和物理地址的页内偏移值一定是一样的。
7. 若父进程在子进程退出前就提前退出时，子进程将成为僵尸进程。
8. 不同进程的虚拟页可通过页表映射到同一物理帧，以节约物理内存，并实现高效的数据共享。
9. RISC-V CPU从用户态（U-Mode）陷入内核态（S-Mode）时，硬件直接把陷入前的用户态代码执行地址保存到sepc寄存器, 并会跳转到stvec寄存器保存的地址执行。
10. 链接器（linker）的一个主要工作是把多个机器码目标文件合并成可执行文件。
11. 对比宏内核架构和微内核架构的操作系统，宏内核架构在性能上有优势，在安全性上有劣势。
12. 在RISC-V架构中，U-mode的用户程序和S-mode 的内核代码可以通过修改 uatp和 satp 寄存器来切换用户程序的页表和内核的页表。
13. 分页机制能消除外碎片，但页面内部没有用满的那部分空间就成了内碎片。
14. 反置页表以物理页框号为索引进行查找：为每个物理帧维护条目，再结合进程标识、哈希方法来反查对应虚拟页。
15. 多级页表按需分配各级页表节点，不用的虚拟地址范围就不分配页表空间，所以相对于单级页表，一定能节省页表所占物理内存。

二、选择题（14分）

1. 从资源管理视角看，下列关于操作系统作用的表述，最恰当的是：
 - A. 负责编译用户程序
 - B. 负责管理 CPU、内存、文件、外设等
 - C. 负责管理用户界面
 - D. 不介入资源分配
2. 关于并发（concurrency）与并行（parallelism），下列说法错误的是：
 - A. 并发是指多个任务在单核处理器上交替执行
 - B. 并行是指多个任务在同一时刻各占一个物理处理器核执行
 - C. 多核处理器上可同时存在并行与并发
 - D. 硬件中断机制是支持并行与并发的充要条件
3. 在 uCore/rCore 实验中，多个用户程序可以在同一个CPU上交替运行，它们共用一块物理内存，每个程序都觉得自己独占了整个地址空间，而且每次运行同一个程序的调度顺序不完全一样。这个场景分别体现了操作系统的哪些基本特征？
 - A. 并发、共享、虚拟、异步
 - B. 并行、独占、虚拟、同步
 - C. 并发、独占、物理、同步
 - D. 并发、共享、物理、异步
4. 在某进程的一次 exec 系统调用前后，进程不可能改变的是：
 - A. PID
 - B. 栈指针
 - C. 进程名字
 - D. 程序计数器
5. RISC-V 64处理器的Sv39分页模式下，每级页表里最多有多少个页表项？
 - A. 256
 - B. 512
 - C. 1024
 - D. 4096
6. uCore/rCore 实验里，用户态程序靠哪条指令陷入内核态来发起系统调用？
 - A. sret
 - B. ecall
 - C. mret
 - D. sfence.vma
7. 在可抢占的分时系统中，下面哪个事件会导致“运行态 → 就绪态”而不是“运行态 → 等待态”
 - A. 基于中断响应的阻塞式 read系统调用等待键盘输入
 - B. 父进程执行wait系统调用并成功回收子进程
 - C. 进程执行exit系统调用
 - D. 时间片耗尽触发时钟中断

三、简答题（71分）

1. （20分）

某同学A在 rCore-Tutorial / uCore-Tutorial 的 ch1 实验环境中修改并编译生成内核代码 os.elf 和 os.bin 后，用 QEMU 启动时无任何串口输出。启动命令如下：

```
qemu-system-riscv64 -machine virt -nographic -bios /BIOS_BIN_DIR/rustsbi-qemu.bin -device loader, file=/OS_BIN_DIR/os.bin,addr=0x80200000
```

他未改动 entry.asm/entry.S 中的入口汇编，但记得曾调整过 linker.ld/kernel.ld。他换了不同版本的QEMU，但并没有解决问题。（注：“X/Y”表述基于 Rust/C 实验的相关部分）。以下是该同学重新编译内核后，通过 riscv64-unknown-elf-objdump 工具得到的输出片段：

```
$ riscv64-unknown-elf-objdump -h os.elf

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .text          0000a000  000000008020a000  000000008020a000  0000a000  2^12
  1 .rodata        00001000  0000000080214000  0000000080214000  00014000  2^12
  2 .data          00002000  0000000080215000  0000000080215000  00015000  2^12
  3 .bss           00004000  0000000080217000  0000000080217000  00017000  2^12

$ riscv64-unknown-elf-objdump -f os.elf

architecture: riscv64, flags 0x00000112:
start address 0x000000008020a000
```

注：riscv64-unknown-elf-objdump的参数含义

-h, --[section-]headers Display the contents of the section headers
-f, --file-headers Display the contents of the overall file header

请回答以下问题：

1. os.bin和os.elf有何区别？os.bin的首字节会被放在QEMU模拟的物理内存的地址是？根据riscv64-unknown-elf-objdump工具输出内容，os.elf文件指出的入口地址是多少？
2. .text, .rodata, .data, .bss的含义是？它们的可读、可写、可执行的属性是？
3. 请推测该同学在 linker.ld 中改错了哪里，并说明该错误如何反映在 riscv64-unknown-elf-objdump工具的输出上，并解释运行时为何会出现“无输出且似死机”现象。

2. （18分）

某同学A尝试在 rCore-Tutorial / uCore-Tutorial ch4实验环境中分别修改内核代码：1）内核栈包含了内核中的函数调用栈信息和 Trap 上下文等信息；2）在进程退出后，内核会调用页表清理函数，该函数对该进程的 Sv39 三级页表递归回收，每层递归在栈上分配 1024 字节临时缓冲区，临时保存当前页表项内容，并执行相关操作来回收进程所占用页表资源；3）实现了 sys_write 系统调用（简化代码如下）：

```

rCore 版本:
fn sys_write(fd: usize, buf: *const u8, len: usize) -> isize {
    let mut kernel_buf = [0u8; 1024];
    let copy_len = len.min(1024);
    unsafe {
        // 未校验 buf 是否为合法用户地址, 直接解引用拷贝
        core::ptr::copy_nonoverlapping(buf, kernel_buf.as_mut_ptr(),
copy_len);
    }
    // ... 后续处理省略 ...
    0
}

uCore 版本
int sys_write(int fd, void *buf, size_t len) {
    char kernel_buf[1024];
    size_t copy_len = len < 1024 ? len : 1024;
    // 未校验 buf 是否为合法用户地址, 直接解引用拷贝
    memcpy(kernel_buf, buf, copy_len);
    // ... 后续处理省略 ...
    return 0;
}

```

某用户程序调用 write 时传入的参数buf 的值为 0x0（内核未映射0x0虚拟地址）。运行后，系统没有按预期返回错误码，而是 panic 并打印 kernel stack overflow。该同学A经过代码分析，发现用户程序传了一个未映射地址进去。但他不明白，一个未检查用户传入地址是否为合法地址的bug，为何会造成内核栈溢出。于是他继续代码分析和调试，发现了如下代码信息：

1. 该内核设置的内核栈大小为 4KB，且内核处理过程中自始至终都使用该内核栈。
2. 该内核的内核态 trap 处理程序中设置了跟用户态 trap 处理程序类似的保存上下文和异常/错误分发逻辑，且该内核的异常/错误分发逻辑中检查到 scause 寄存器的错误类型为 Load Page Fault（读数据错误异常）时，会直接终止进程。

根据以上信息，他通过多次调试，初步分析出系统panic 并打印 kernel stack overflow 现象的根本原因。请回答以下问题：

1. 从用户程序发起write系统调用到 kernel stack overflow结束，内核的执行路径主要包含哪些控制逻辑？内核栈上分别压入了哪些内容？请计算总的内存占用是否超过内核栈大小？
2. 该同学A认为修这个 bug，只需要系统调用入口检查参数即可。同学B听说了这个 bug 之后，建议增大内核栈空间即可。同学C则建议建立内核中断栈，并在进入异常/错误分发逻辑时使用这个专门的内核中断栈。请分别分析这三个建议能否避免这个问题以后再次出现？

注1：scause是S模式陷入原因寄存器，当任何特权模式（U-mode 或 S-mode）下发生异常或中断，并且最终要进入 S-mode 处理时，RISC-V处理器会自动将本次陷阱（Trap）的类型（中断/异常）和具体原因编码写入 scause。

3. (15分)

某同学A在 rCore-Tutorial / uCore-Tutorial ch4实验环境中修改了内核代码，并测试用户程序。该用户程序通过 mmap 函数申请了一段匿名内存，起始地址为 0x10000，大小为 8192 字节。修改后的内核代码为该匿名内存创建了虚拟内存区域（课件中的 MemorySet::MapArea：一段虚拟地址连续的虚存内存，即Virtual Memory Area, VMA），VMA的权限标记为可读可写。随后，程序执行了以下操作：

uCore 版本

memset((void *)0x10000, 0, 8192);

rCore 版本

core::ptr::write_bytes(0x10000 as *mut u8, 0u8, 8192);

该操作从 0x10000 开始，由低地址向高地址连续写入 8192 字节的0。用户程序运行后，进程并未完成清零，而是被内核直接终止。该同学在debug的时候，查看到最后一次错误的 scause 寄存器的值为15（Store/AMO Page Fault，写数据或原子操作页错误的异常），他很困惑：明明已经通过 mmap 函数申请了合法区域，而且在内核中实现了明确的懒分配（按需分页）策略，为何写入还会失败，并造成内核直接终止？
注：懒分配策略是指：若发生异常的虚拟地址在某个 VMA 内，且对应页表项（PTE）的 V=0，则执行懒分配处理：分配物理页，建立可读可写映射（V=1, R=1, W=1），恢复进程。否则在其他任何情况，内核直接终止进程。

请回答如下问题：

- 1. 内核尽可能推迟真正的内存分配和复制操作的优化策略，除了懒分配策略，还有哪些新策略？请说明新策略的大致思想。
- 2. 通过 satp 寄存器得知，当前进程的根（一级）页表的物理地址为 0x80400000。该进程的二级页表和三级页表的物理地址分别是？
- 3. 请分析用户程序的 memset 执行过程中，最终触发 scause 为15的异常错误，并被内核直接终止的原因是什么？请描述从异常产生到操作系统处理的完整流程，按顺序列出涉及的硬件自动操作和软件处理步骤。
- 4. 如果用户程序的 memset 能正确执行，内核还需要实现什么功能？为什么？

当前进程的各级页表的部分内容如下：

一级页表：

偏移	内容（十六进制）
0x000	0x000000002005C001

二级页表：

偏移	内容（十六进制）
0x000	0x0000000020094001

三级页表：

偏移	内容（十六进制）
0x080	0x00000000c0000013
0x088	0x0000000000000000

在 RISC-V Sv39 模式下，39 位虚拟地址划分为：

位域	名称	描述
38:30	VPN[2]	一级页表索引（9 位）
29:21	VPN[1]	二级页表索引（9 位）
20:12	VPN[0]	三级页表索引（9 位）
11:0	Page Offset	页内偏移（12 位）

一个页表项（PTE）为 64 位（8 字节），结构如下：

位域	名称	描述
63:54	Reserved	保留
53:10	PPN[2:0]	物理页号，共 44 位
9:8	RSW	保留给操作系统软件使用
7	D	脏位
6	A	访问位
5	G	全局映射
4	U	用户态可访问
3	X	可执行
2	W	可写
1	R	可读
0	V	Valid（有效位）

4. （18分）

下面的C程序可以在 Linux 下运行。假设所有 fork 和 wait 调用都能成功执行。

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int x = 1;
    if (fork() > 0) {
        x = x + 1;
        wait(NULL);
    } else {
        x = x + 2;
        if (fork() == 0) {
            x = x + 3;
        }
    }
    printf("%d\n", x);
    return 0;
}
```

请回答如下问题：

- 1、这个程序一共产生了多少个进程（包括一开始执行 main 的那个进程）？请画出进程之间的父子关系。
- 2、程序可能输出哪些内容？请列出所有可能的输出序列，说明理由。
- 3、执行fork后，父进程和子进程各自的变量x是存放在同一块物理内存中吗？请解释fork处理此事的机制。
- 4、执行fork后，子进程是怎么做到从fork返回处拿到返回值0的？请说明fork处理此事的机制。